Cache coherence in shared memory system

Cache Coherence

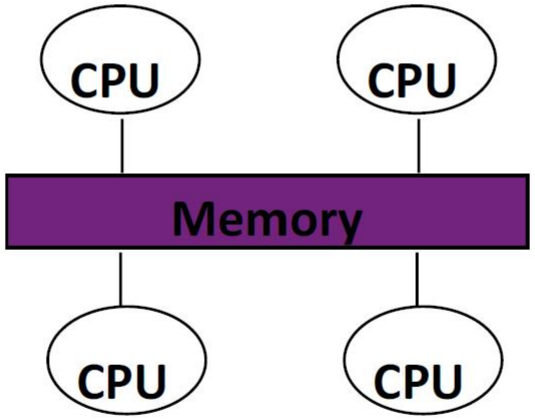
When multiple processors have **their own cache**, they may keep copies of the same data. If one processor changes the data, the other processors' cached copies must also be updated or invalidated. This is called **cache coherence**.

Shared Memory Systems

- All processes have access to the same address space
  - E.g. PC with more than one processor
- Data exchange between processes by writing/reading shared variables
  - Shared memory systems are easy to program
  - Current standard in scientific programming: OpenMP
- Two versions of shared memory systems available today
  - Centralized Shared Memory Architectures
  - Distributed Shared Memory architectures

Centralized Shared Memory Architecture

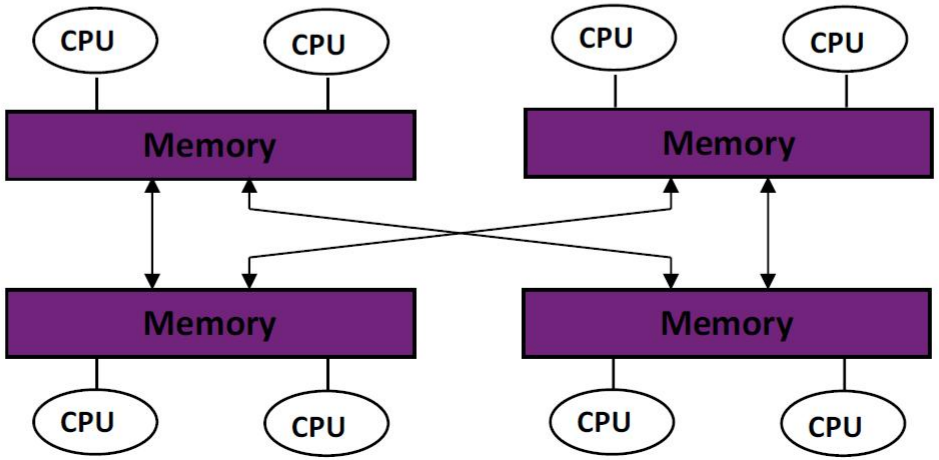
- Also referred to as Symmetric Multi-Processors (SMP)
- All processors share the same physical main memory



- Memory bandwidth per processor is limiting factor for this type of architecture
- Typical size: 2-32 processors

Distributed Shared Memory Architectures

- Also referred to as Non-Uniform Memory Architectures (NUMA)
- Some memory is closer to a certain processor than other memory
  - The whole memory is still addressable from all processors
  - Depending on what data item a processor retrieves, the access time might vary strongly



NUMA Architectures (II)

- Reduces the memory bottleneck compared to SMPs
- More difficult to program efficiently
  - E.g. first touch policy: data item will be located in the memory of the processor which uses a data item first

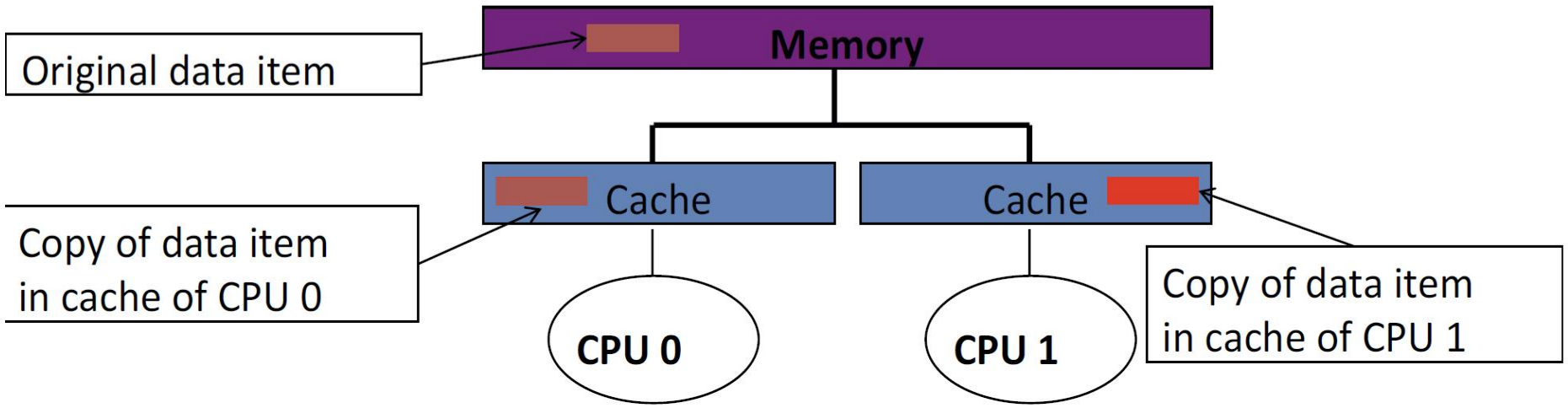
Whoever touches it first, keeps it

- To reduce effects of non-uniform memory access, caches are often used
  - ccNUMA: cache-coherent non-uniform memory access architectures
- Largest example as of today: SGI Origin with 512 processors


S|A
www.semiaccurate.com

Cache Coherence

- Real-world shared memory systems have caches between memory and CPU
- Copies of a single data item can exist in multiple caches
- Modification of a shared data item by one CPU leads to outdated copies in the cache of another CPU



Cache Coherence (II)

- Typical solution:
  - Caches keep track on whether a data item is shared between multiple processes
  - Upon modification of a shared data item, 'notification' of other caches has to occur
  - Other caches will have to reload the shared data item on the next access into their cache
- Cache coherence is only an issue in case multiple tasks access the same item
  - Multiple threads
  - Multiple processes have a joint shared memory segment
  - Process is being migrated from one CPU to another

Cache Coherence Protocols

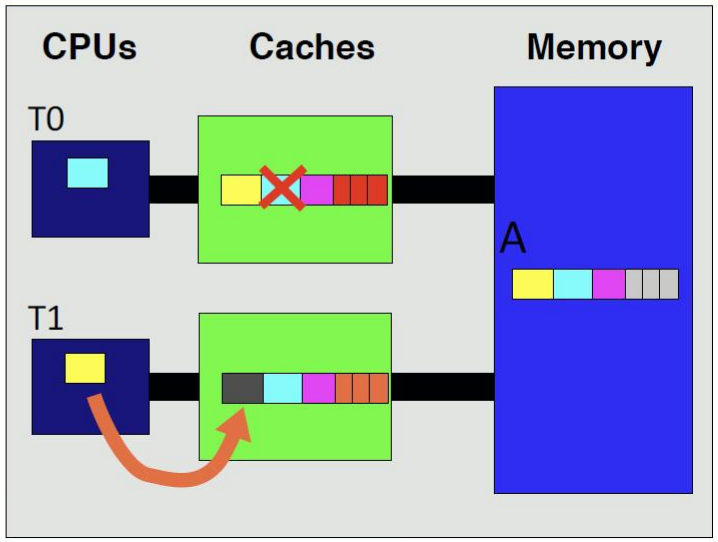
- Snooping Protocols
  - Send all requests for data to all processors
  - Processors snoop a bus to see if they have a copy and respond accordingly
  - Requires broadcast, since caching information is at processors
  - Works well with bus (natural broadcast medium)
  - Dominates for centralized shared memory machines
- Directory-Based Protocols
  - Keep track of what is being shared in centralized location
  - Distributed memory => distributed directory for scalability (avoids bottlenecks)
  - Send point-to-point requests to processors via network
  - Scales better than Snooping
  - Commonly used for distributed shared memory machines

Categories of Cache Misses

- Up to now:
  - Compulsory Misses: first access to a block cannot be in the cache (cold start misses)
  - Capacity Misses: cache cannot contain all blocks required for the execution
  - Conflict Misses: cache block has to be discarded because of block replacement strategy
- In multi-processor systems:
  - Coherence Misses: cache block has to be discarded because another processor modified the content
    - true sharing miss: another processor modified the content of the request element
    - false sharing miss: another processor invalidated the block, although the actual item of interest is unchanged.

False Sharing in OpenMP

- False sharing
  - When at least one thread write to a cache line while others access it
    - Thread 0: = A[1] (read)
    - Thread 1: A[0] = … (write)
- Solution: use array padding



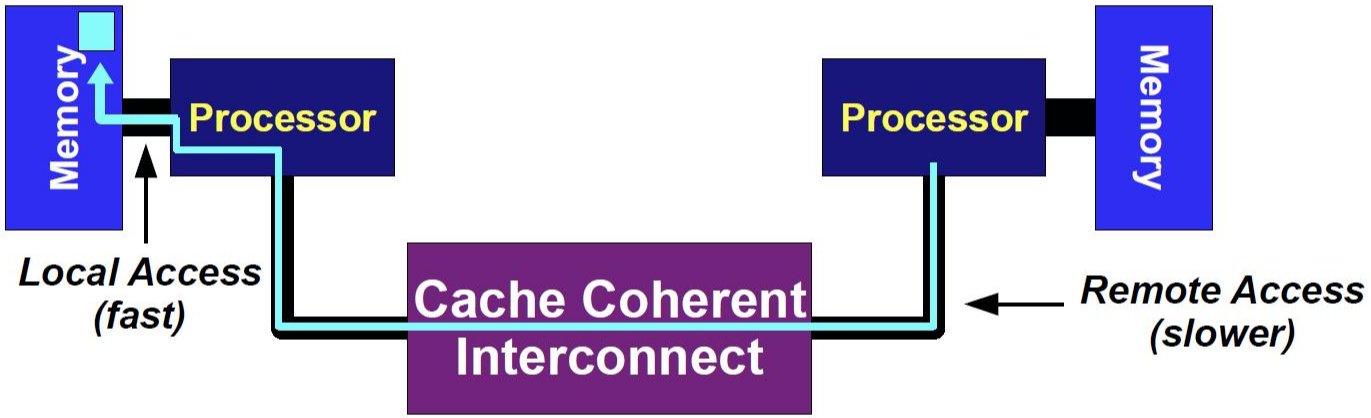
We add empty, useless space

```
int a[max_threads];
#pragma omp parallel for schedule(static,1)
for(int i=0; i<max_threads; i++)
    a[i] +=i;
```

```
int a[max_threads][cache_line_size];
#pragma omp parallel for schedule(static,1)
for(int i=0; i<max_threads; i++)
    a[i][0] +=i;
```

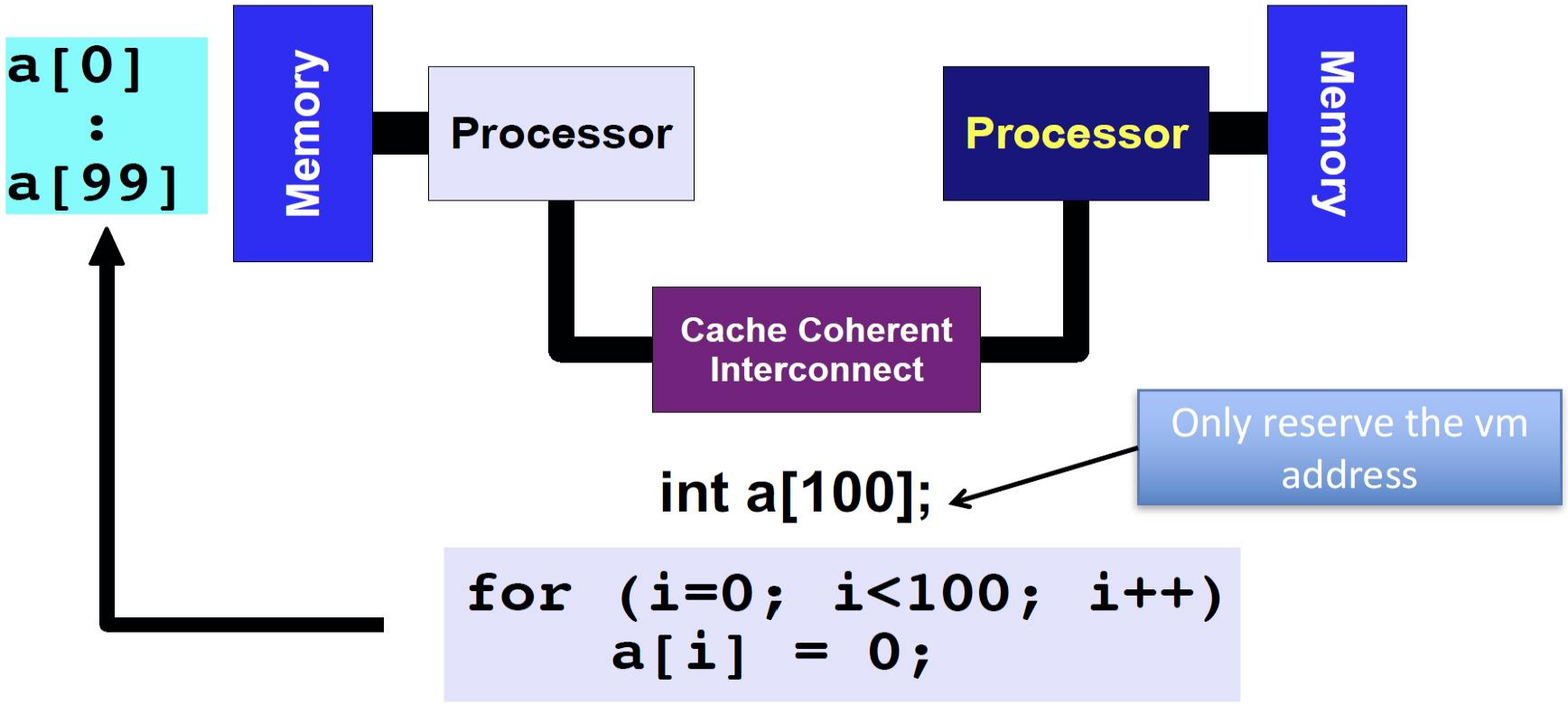
NUMA and First Touch Policy

- Data placement policy on NUMA architectures



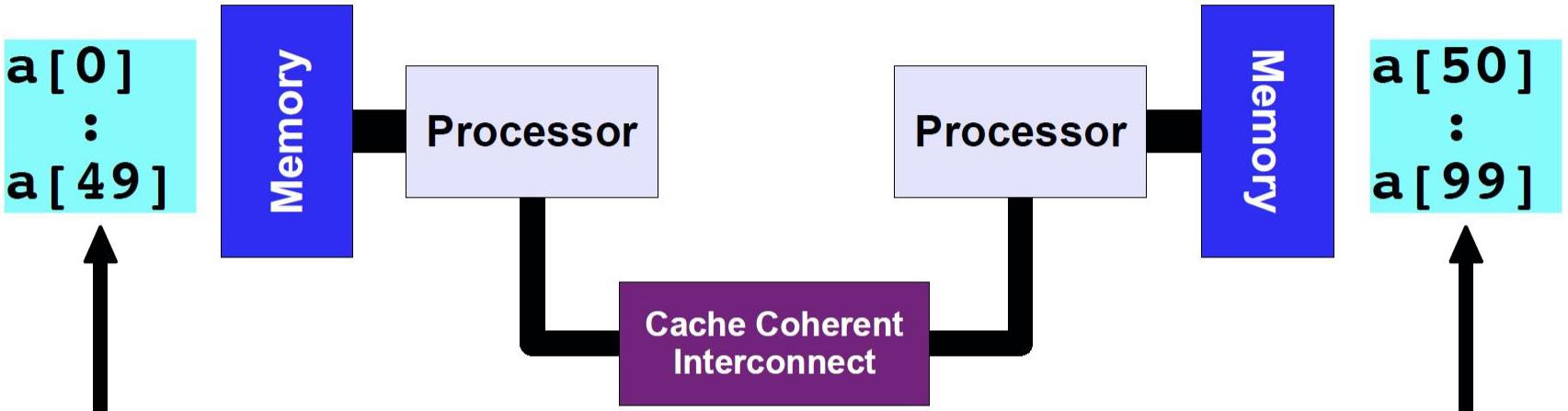
- First Touch Policy
  - The process that first touches a page of memory causes that page to be allocated in the node on which the process is running

NUMA First-touch Placement/1



First Touch
All array elements are in the memory of the processor executing this thread

NUMA First-touch Placement/2




```
#pragma omp parallel for num_threads(2)
for (i=0; i<100; i++)
    a[i] = 0;
```

First Touch

Both memories each have "their half" of the array

Work with First-Touch in OpenMP

- First-touch in practice
  - Initialize data consistently with the computations

To make the code fast, programmers use the same parallel loop for initialization and computation. This guarantees that each chef initializes and then uses their own half of the data, keeping everything on their local shelf and avoiding the slow walk across the kitchen.

```
#pragma omp parallel for
for(i=0; i<N; i++) {
   a[i] = 0.0; b[i] = 0.0 ; c[i] = 0.0;
}
readfile(a,b,c);

#pragma omp parallel for
for(i=0; i<N; i++) {
    a[i] = b[i] + c[i];
}
```

Concluding Observations

- Programmer can optimize for cache performance
  - How data structures are organized
  - How data are accessed
    - Nested loop structure
    - Blocking is a general technique
- All systems favor “cache friendly code”
  - Getting absolute optimum performance is very platform specific
    - Cache sizes, line sizes, associativities, etc.
  - Can get most of the advantage with generic code
    - Keep working set reasonably small (temporal locality)
    - Use small strides (spatial locality)
  - Work with cache coherence protocol and NUMA first touch policy